

2} Arquitectura y concurrencia

1) Patrón SSOT:

SSOT centraliza el acceso a los datos de la aplicación, donde la UI solo consulta al repositorio.

El repositorio usa cache, implementado estrategias de almacenamiento, la UI permanece desconocida del proceso de sincronización

2} Corrutinas y Concurrencias

El `viewModelScope` está vinculado al `viewModel`, evita fugas de memoria automáticamente.

Si el usuario cierra la pantalla, el `viewModel` se destruye y el `viewModelScope` se cancela.

3} Flujos reactivos

- Flow no ejecuta código ni datos hasta que haya un recolector activo.
- Stateflow está siempre activo emitiendo su estado.
- Por eso se prefiere este último.

4} Inyección de dependencias

- DI Manual: Se ejecuta al iniciar la app, al instanciar contenedores, los objetos se vuelven accesibles de forma global.
- Permitiendo la inicialización tardía usando el lazy en objetos pesados.

3) Completar.

- 1) El flujo de datos que emite cada vez que una tabla Room cambia se implementa usando el tipo Flow.
- 2) El operador Dispatchers permite ejecutar tareas pesadas fuera del Main Thread, específicamente optimizado para tareas de CPU.
- 3) Para que un servicio en primer plano de la ubicación sea válido en Android 10+ debe declararse en el manifiesto con el tipo location.
- 4) La función applicationContext de la clase Application es la que garantiza que los componentes comparten la misma instancia.
- 5) El componente que permite transformar una API basada en callbacks antiguos en una función suspendible moderna se llama Flow.
- 6) El archivo de metadatos que describe la estructura de la app al SO se llama AndroidManifest.xml.
- 7) En la arquitectura AOSP, la capa que actúa como puente estándar entre el framework y los controladores se denomina HAL o Capa de Abstracción Hardware.
- 8) El operador de flow que permite combinar múltiples fuentes (como GPS y sensores) para emitir un estado unificado es combine.

4) Desarrollo de código

1) Repositorio y DI

```
class UsuarioRepository( val usuarioDao: UsuarioDao)
```

```
class MiAplicacion : Application() {
```

```
    val database by lazy { MiBase.getInstance(this) }
```

```
    val usuarioDao by lazy { database.usuarioDao() }
```

```
    val usuarioRepository by lazy {
        UsuarioRepository(usuarioDao)
    }
```

```
}
```

2) Logica Dinamica Identidad

```
→ a) data class Usuario( val nombre: String?,
                        val apellidos: String?,
                        val edad: Int? )
```

```
// en una función
```

```
→ b) val miUser = Usuario(
    nombre = "Juan",
    apellidos = "Flores Huaman",
    edad = 20
)
```

```
→ c) ye)
```

```
    val letraNombre = miUser.nombre?.replace(" ", "").length?: 0
    val letraApellidos = miUser.apellidos?.replace(" ", "").length?: 0
    val puntajeId = letraNombre + letraApellidos
```

```
    val nombreTodo = "${miUser.nombre?: ""} ${miUser.apellidos?: ""}.timu)
```

```
→ d)
```

```
    println("Usuario: $nombreTodo, tu puntaje de identidad es: $puntajeId")
```


5>

Arquitectura AOSP

1>

